

From Code to Coverage

A Detection Engineer's Journey Through the LDAP Wilderness

```
$ # Six months. Nine blind spots. One detection stack.  
$ wc -l ~/endpoints.csv          # MDR scale  
  ~5,000,000  
$ ls ~/sigma/attempts/ | wc -l  # too many to count  
$ grep -rc 'blind spot' ~/blog/  
  9
```

[speaker] Andrew Schwartz

[handle] @4ndr3w6S



```
$ ldapsearch -x -H ldap://lab.local \  
-b 'ou=speakers,dc=lab,dc=local' \  
'(SAMAccountName=4ndr3w6S)'
```

```
dn: CN=Andrew Schwartz,OU=Speakers,DC=lab,DC=local  
displayName: Andrew Schwartz  
SAMAccountName: 4ndr3w6S  
title: Principal Detection Engineer  
company: Huntress  
description: AD tradecraft · LDAP · Sigma  
businessCategory: Detection · Kerberos research  
memberOf: CN=Detection Engineers  
          CN=Always Thinking Purple  
comment: Diamond Ticket · DACL · dMSA Ouroboros  
info: Tottenham Hotspur · COYS · chess  
drink: old fashioned // boulevardier
```

```
# Result: 1 entry returned ( visited: many )
```

Six months. 5M endpoints. No baseline. The stack that finally worked.

Why I wrote dozens of broken rules

The failure story

The constraint: *5M endpoints at MDR pace — every rule has to generalize.*

Signatures: a clean Impacket rule, tested — zero hits. The DC rewrote it first.

ADWS: every Get-ADUser query logs ::1 — never the attacker.

cLDAP: ldapnomnom sprayed usernames — zero Event 1644.

Obfuscation: ~70% of MaLDAPtive techniques produce no 1644 at all.

Stats: then one z-score caught tools I had never seen.

The fix: one source is never enough — signatures + behavior + stats + ETW.

What you leave with

- ▶ Detection patterns for Impacket, BloodHound, SOAPHound
- ▶ Whitespace-agnostic detection patterns
- ▶ SDFlags + (!FALSE) behavioral hunting
- ▶ Statistical detection that catches unknown tools
- ▶ ADWS source IP attribution (Event 5156 ↔ 1644)
- ▶ netlogon.log for LDAP Ping detection
- ▶ LDAPMon (client-side ETW) for obfuscation
- ▶ Where each log source is blind — and the fix

LDAP recon is every attack's first move

BloodHound

SharpHound

Impacket

SOAPHound

ldapnomnom

MaLDAPtive

50+

valid whitespace variants of one filter

::1

source IP Event 1644 shows for every ADWS query

0

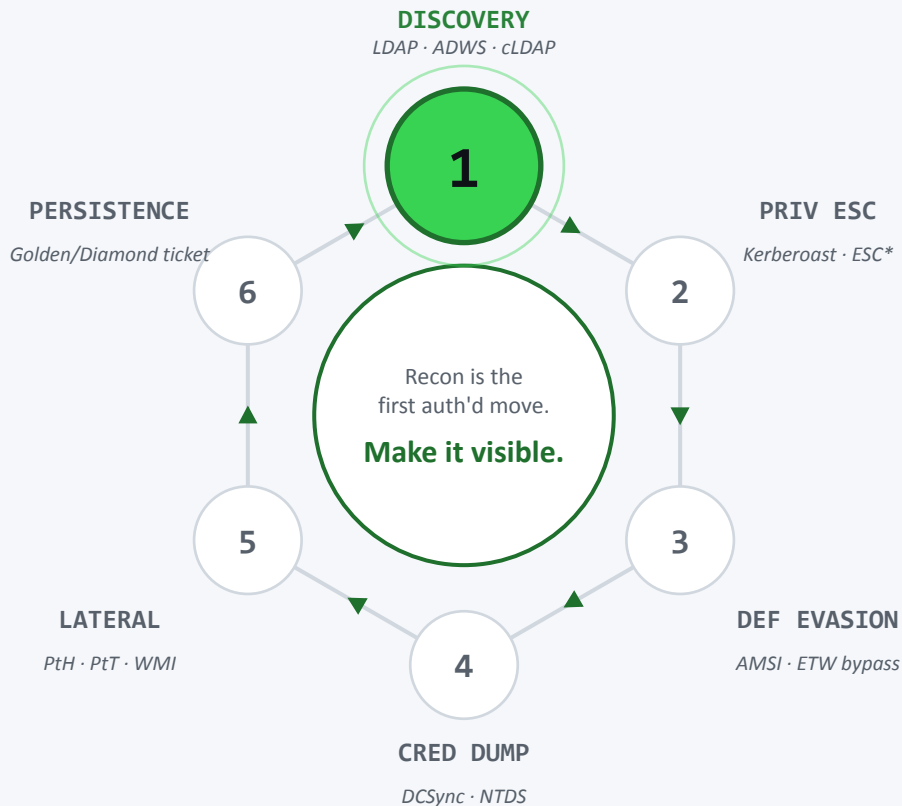
Event 1644 from an ldapnomnom username spray

~70%

of obfuscation techniques produce NO Event 1644

Signatures break the moment you meet a real attacker. You need a different mental model.

■ // THE AD KILL CHAIN • WHERE THIS TALK LIVES



Out of scope: Kerberoasting, ADCS abuse, delegation paths, DCSync, lateral movement, persistence — *all depend on recon working first.*

Reference: github.com/infosecninja/AD-Attack-Defense

Three arcs. Nine parts. One thesis.

Talk order follows the attacker's path, not the release calendar.

ARC 1 — SIGNATURE DETECTION

PUBLISHED

What tools send vs. what gets logged.

-
- 01 OID Transformations
 - 02 Whitespace & Formatting
 - 03 SDFlags Hidden Params
 - 04 SOAPHound (!FALSE)

ARC 2 — BLIND SPOTS

IN PROGRESS

Where Event 1644 fundamentally fails you.

-
- 5A ADWS: the ::1 Blind Spot
 - 5B Event 5156 — Source IP Attribution
 - 06 LDAP Ping / Netlogon

ARC 3 — BEYOND SIGNATURES

IN PROGRESS

Breaks even a perfect Event 1644 setup.

-
- 07 Statistics / Volumetric
 - 08 Low & Slow Evasion
 - 09 Obfuscation / LDAPMon

One thesis: Event 1644 alone is not enough. *You need the forest, not the tree.*

Before anything else: turn on Event 1644

```
# Two registry keys. One reboot-free.  
# Run as admin on the DC (EARTH-DC).  
  
$diag = 'HKLM:\SYSTEM\CurrentControlSet\  
        Services\NTDS\Diagnostics'  
$param = 'HKLM:\SYSTEM\CurrentControlSet\  
        Services\NTDS\Parameters'  
  
# 1. Turn on diagnostic logging (level 5)  
Set-ItemProperty $diag `   
    -Name '15 Field Engineering' `   
    -Value 5 -Type DWord  
  
# 2. Set the expensive-search threshold  
Set-ItemProperty $param `   
    -Name 'Expensive Search Results Threshold' `   
    -Value 1 -Type DWord  
  
# Lab: threshold = 1 (log everything)  
# Prod: start at INVESTIGATE tier (slide 43)
```

What you get

Event 1644 in **Directory Service** log.
Fires for every LDAP search that crosses the threshold.
Level 5 is REQUIRED — lower levels won't emit 1644.



CPU & volume caveats

- Level 5 + threshold=1 = every query logged.
- Measure EPS in lab first. Busy production DCs log a lot.
- Lower thresholds = more load on busy DCs. Tune threshold to your floor.
- **Not a CIS/NIST required control — it's a diagnostic feature.**

Everything downstream assumes 1644 is on. If it isn't, the rest is theater.

Raw Event 1644 – from my DC, right now

Event 1644 • Directory Service

Internal event: A client issued a search operation with the following options.

Client:

[::1]:59435

Starting node:

DC=marvel,DC=local

Filter:

(! (FALSE))

Search scope:

subtree

Attribute selection:

objectSid,objectGUID,
distinguishedName,objectClass

Server controls:

SDflags:0x7;

Visited entries:

3815

Returned entries:

334

Used indexes:

DNT_index:2584:N;

Pages referenced:

23386

Pages read from disk:

0

Pages preread from disk:

0

Clean pages modified:

0

Dirty pages modified:

0

Search time (ms):

47

Attributes Preventing Optimization:

none

User:

MARVEL\loki

Read every column

CLIENT

[::1] — query came via ADWS,
not the real attacker IP.

FILTER

(! (FALSE)): no legit app sends it.
Chapter 04 fingerprint.

ATTRIBUTES

Only 4 attributes. No nTSecurityDescriptor.
...yet SDflags:0x7 is set anyway.

CONTROLS

SDflags:0x7 = the SOAPHound family.
Chapter 03 detection.

STATS

Visited 3815 / Returned 334 / Pages.
Chapter 07 math fodder.

Note: Client shows [::1] — *this query came via ADWS. That's the Chapter 5A problem.*

CHAPTER 01

OID Transformations

Your rule never had a chance. The DC rewrote the query before logging.

What the attacker sends vs. what you log

IMPACKET SENDS

```
(&(|(UserAccountControl:  
  1.2.840.113556.1.4.803:=16777216)  
(UserAccountControl:  
  1.2.840.113556.1.4.803:  
  =524288) ...)
```



EVENT 1644 LOGS

```
( & ( | (userAccountControl&16777216)  
      (userAccountControl&524288)  
      ... ) ... )
```

*LDAP RFC 4515 extensible match
1.2.840.113556.1.4.803 = bitwise AND*

*OID has been normalized to the & operator
Your regex for "1.2.840" matches nothing.*

LESSON: Write rules against the logged form, not the wire form. Know both.

The transformation OIDs

OID	Operator	Appears in 1644 as
1.2.840.113556.1.4.803	Bitwise AND	<code>attr&value</code>
1.2.840.113556.1.4.804	Bitwise OR	<code>attr value</code>

WORKING SIGMA RULE

```
title: Impacket LDAP Recon
logsource:
  product: windows
  service: directory-service
detection:
  sel_bitwise:
    event.code: 1644
    winlog.event_data.LDAPFilter|re:
      '&d{3,}'
  condition: sel_bitwise
level: medium

# matches '&524288' etc.
# not '1.2.840.113556...'
```

CHAPTER 02

Whitespace & Formatting

Fifty ways to write the same query. One rule to match them all.

Same query. Seven of fifty valid forms.

#01 (&(objectClass=user)(cn=admin))

#02 (& (objectClass=user) (cn=admin))

#03 (& (objectClass=user) (cn=admin))

#04 (&(ObjectClass=user)(CN=admin))

#05 (&(objectClass=USER)(cn=ADMIN))

#06 (&\n(objectClass=user)\n(cn=admin))

#07 (&(objectClass=\75ser)(cn=admin))

All seven: the same query to AD. None is logged as sent: 1644 writes the DC's own canonical form.

Normalize, then match

NAIVE — BREAKS ON FORMATTING

```
detection:
  sel:
    winlog.event_data.LDAPFilter|contains:
      '(&(objectClass=user)(cn='
  condition: sel
```

CORRECT — NORMALIZE FIRST

```
detection:
  sel:
    winlog.event_data.LDAPFilter|re|i:
      '\(\\s*&\\s*\\(\\s*objectclass'
  condition: sel
```

The performance tradeoff

- Regex costs more CPU than |contains on high-volume 1644 data
- Pre-filter with event.code and an anchor token (e.g. 'objectclass'), then regex
- Measure events/sec on YOUR DC before shipping — regex on 50k EPS hurts

CHAPTER 03

SDFlags

The server-side control BloodHound asks for. Detection gold.

SDFlags: the BloodHound tell

What it is

Server-side LDAP control (OID 1.2.840.113556.1.4.801).

Tells AD which parts of nTSecurityDescriptor to return:

0x01 owner

0x02 group

0x04 DACL ← access control

0x08 SACL ← audit

Flags add: 0x5 = owner + DACL · 0x7 = owner + group + DACL

SharpHound asks for 0x4 or 0x5. SoaPy and SOAPHound ask for 0x7.

Over ADWS, nTSecurityDescriptor is impossible WITHOUT this control — its presence proves SDFlags was set.

What the event looks like

```
Event 1644 · excerpt (--bhdump)
Client:
[::1]:59435
Attribute selection:
name,sAMAccountName,cn,...,
nTSecurityDescriptor,...,objectClass
Server controls:
SDflags:0x7;
Visited entries:
3815
Returned entries:
334
```

Why attackers WANT nTSecurityDescriptor

What's inside it

The security descriptor for every AD object:

Owner — who owns the object

Group — primary group

DACL — WHO can do WHAT

SACL — audit (rarely set)

*The DACL is the attack graph.
Every ACE = a potential edge
in BloodHound's path-finder.*

Why they want it

Every ACE = a possible privesc.

- GenericAll / WriteDACL
- WriteOwner / ForceChangePwd
- AddSelf / AddMember
- RBCD source SID writes
- AdminSDHolder bypass

*Reading the DACL is how
they pick their next move.*

Why SDFlags is forced

**Query nTSecurityDescriptor
without SDFlags:**

1. AD tries all 4 parts (default)
2. SACL needs SeSecurityPriv
3. No SeSecurityPrivilege → no SACL
4. Over ADWS: attribute dropped

*SDFlags = 0x4, 0x5 or 0x7
skips SACL → success.
So they always set it.*

Legitimate admins query specific objects. Attackers query everything. SDFlags presence is the difference.

What the rule actually looks like

```
title: BloodHound Family ACL Enumeration
id: 8fa99195-83b7-4c0e-b8b9-4026b444ce19
status: stable
author: Andrew Schwartz
tags:
  - attack.discovery
  - attack.t1087.002
  - tool.sharphound
  - tool.soapy
logsource:
  product: windows
  service: directory-service
detection:
  selection:
    event.code: 1644
    winlog.event_data.SearchFlags|contains:
      - 'SDFlags:0x4;'
      - 'SDFlags:0x5;'
      - 'SDFlags:0x7;'
    winlog.event_data.AttributesRequested|contains: 'nTSecurityDescriptor'
    winlog.event_data.SearchScope: 'subtree'
  filter_replication:
    winlog.event_data.SearchFlags|contains: 'return_deleted'
  condition: selection and not filter_replication
level: high
```

What it catches

- SharpHound (default + DCOOnly)
- BloodHound.py
- SOAPHound (ACL runs)
- Manual ACL recon

→ 0x4, 0x5, 0x7 catches all.

Where it lives

Huntress production rule.

Plus 6 more in the repo:

- Generic SDFlags
- ADCS template recon
- Whitespace-agnostic OID

Not pseudocode. Not pseudo-Sigma. This is the rule.

CHAPTER 04

SOAPHound

A clever (!FALSE) optimization that becomes a fingerprint.

The (!FALSE) fingerprint

The clever-to-a-fault optimization

SOAPHound enumerates with `(!soaphound=*)` — a presence check for an attribute that doesn't exist. Since nothing ever has `soaphound=`, AD's query optimizer collapses the inner expression to `FALSE` and the outer `(!FALSE)` becomes `TRUE` — returning every object.

```
title: SOAPHound (!FALSE) filter
name: soaphound_false
logsource:
  product: windows
  service: directory-service
detection:
  sel:
    event.code: 1644
    winlog.event_data.LDAPFilter|re:
      '\(\\s*!\\s*\\(\\s*FALSE'
  condition: sel
```

```
---
title: SOAPHound (!FALSE) burst
correlation:
  type: event_count
  rules:
    - soaphound_false
  group-by:
    - winlog.event_data.Account
  timespan: 5m
  condition:
    gte: 5
```

Stop hunting names. Hunt behavior.

SIGNATURE – dies on rename

String 'SOAPHound' in filter

Tool name = 'SharpHound'

Specific OID 1.2.840.113556.1.4.803

Query matches known tool fingerprint

PATTERN – survives evasion

(!(nonexistent=*)) with high entry count

SDFlags $\in$ {0x4,0x5,0x7} OR nTSecurityDescriptor in attrs

&d{3,} – any bitwise operator + value

EntriesVisited/EntriesReturned ratio

Same moment – attacker view + DC view

// ATTACKER • SanctuaryII-RTO

```
C:\Users\loki.MARVEL\Desktop> SOAPHound.exe ^
--user loki@marvel.local --password ***** ^
--dc 10.1.1.11 --bhdump -o bh_output --nolaps
```

```
Loading cache from disk
Loaded cache with stats: 264 ID to type mappings.
264 name to SID mappings.
-----
```

```
Gathering AD data
ADWS request with ldapbase (DC=marvel,DC=local),
ldapquery: (!soaphound=*) and ldapproperties:
[name, sAMAccountName, cn, dNSHostName, objectSid,
nTSecurityDescriptor, member, ... gPOptions]
```

Gathering AD data complete

```
-----
Gathering DomainTrusts data
ldapquery: (trustType=*)
Gathering DomainTrusts data complete
```

Output files with prefix "full" -> bh_output\

// DEFENDER • EARTH-DC • Event 1644 (excerpt)

```
Client:
[::1]:59435
Filter:
( ! (FALSE) )
Search scope:
subtree
Attribute selection:
name,sAMAccountName,cn,dNSHostName,
objectSid,objectGUID,primaryGroupID,
distinguishedName,lastLogonTimestamp,
...,sIDHistory,nTSecurityDescriptor,...
Server controls:
SDflags:0x7;
Visited entries:
3815
Returned entries:
334
User:
MARVEL\loki
```

SOAPHound's (!soaphound=*) becomes (!(FALSE)). The fingerprint AD writes for you. *Two sides of the same moment — from my lab.*

CHAPTER 5A

PowerShell → ADWS

Get-ADUser is the attacker's love letter. Event 1644 logs ::1 — not them.

PowerShell → ADWS → ::1 → 1644



"I've not found a way to correlate the originating host whatsoever."

– BloodHound community Slack

...until you look at a different log.

The BloodHound Slack exchange

anon

what's your preferred way of opsec-safe AD enum — ADSI or ADWS?

anon

ADWS. but client always appears as the DC itself — service talks to LDAP on loopback.

anon

I've not found a way to correlate the originating host. alerting is very difficult.

Charlie Clark

surely you could with a network event? 5156?

me

Challenge accepted.

The data was always there. Nobody was correlating it.

CHAPTER 5B

Event 5156 – Port Correlation

Join the external port to the proxied 1644. Source IP restored.

Event 5156 restores the source IP

```
# (1) External 5156 – the real attacker hits ADWS
EventID: 5156 SrcIP: 10.1.1.13 SrcPort: 48594 DstIP: 10.1.1.11 DstPort: 9389
```

```
# (2) Internal 5156 – ADWS proxies to local LDAP
EventID: 5156 SrcIP: ::1 SrcPort: 53149 DstIP: ::1 DstPort: 389
```

```
# (3) 1644 – logs localhost and the proxied source port 53149
EventID: 1644 Client: [::1]:53149 Filter: ( & (objectClass=user) ... )
```

JOIN (2).SrcPort == (3).ClientPort • JOIN (1) ≈ (2) within 60–80 ms



// VIDEO DEMO

Get-ADWSAttribution-dual.ps1

Restore the source IP for ADWS queries — port match + timestamp fallback.

Get-ADWSAttribution-dual.ps1



VIDEO

DEMO 01 • Get-ADWSAttribution-dual.ps1

The event chain – captured, not theorized

SUSPICIOUS ADWS QUERY DETECTED – MARVEL.local (12/15 lab capture)

Source IP: 10.1.1.13 User: MARVEL\loki

Correlation: TIMESTAMP | High (61.6 ms)

[5156] 17:30:08.214 - Security log (WFP Connection)

Src: 10.1.1.13:48594 -> DC:9389 (ADWS)

App: microsoft.activedirectory.webservices.exe

| 46.4 ms

[1138] 17:30:08.260 - Directory Service (LDAP Op Start)

Client: [::1]:53149 User SID: S-1-5-21-...-1105

>> ADWS proxy to LDAP (localhost)

| 15.2 ms

[1644] 17:30:08.275 - Directory Service (Expensive Search)

Client: [::1]:53149 User: MARVEL\loki

Filter: (& (objectClass=user) (objectCategory=...))

Attributes: [all_with_list]nTSecurityDescriptor

Controls: SDflags:0x7 ← full SD (BloodHound)

Stats: Visited 53 / Returned 53 / Pages 1967

[!] Indicators: ACL enumeration, Full SD (BloodHound pattern)

Three events. ~62ms apart. Joined back to 10.1.1.13 / MARVEL\loki. **Not theoretical — this is from my lab.**

BONUS • CH 5B

Multi-Event Correlation

When 1644 isn't enough: stitch 1138, 1139, 5156, and 1644 into one story.

The full event chain

} bonus events on client (optional)

Sysmon 7*	[client†]	Image load of Microsoft.ActiveDirectory.Management.dll (if Sysmon config logs it)
------------------	-----------	---

PS 4104	[client]	PowerShell script block logging: Get-ADUser -Filter *
----------------	----------	---

5156	[client]	Outbound TCP to DC:9389 — DLL connects to ADWS
-------------	----------	--

5156	[DC]	ADWS proxies to ::1:389 — ephemeral port recorded
-------------	------	---

1138	[DC]	LDAP operation START — user, ::1:<port>, ldap_search, sequence #
-------------	------	--

1644	[DC]	LDAP query logged with client=::1:<port>
-------------	------	--

1139	[DC]	LDAP operation COMPLETE — status, same sequence #
-------------	------	---

Correlate by port + user + sequence #. Full tool + source + user + query attribution.

** Sysmon Event 7 fires only if Sysmon config tracks DLL loads (most do not by default). PowerShell 4104 requires script block logging via GPO.*

CHAPTER 06

Ldapnomnom

Event 1644 is blind by design. netlogon.log catches every query.

LDAP Ping is not LDAP search

Regular LDAP Search

- Requires authentication
- Queries the AD directory tree
- Processed by ntdsa.dll in lsass
- **Event 1644 fires**

Used by: BloodHound, SharpHound, Impacket, PowerView

LDAP Ping (cLDAP)

- ANONYMOUS — no auth needed
- Does NOT touch the directory
- Intercepted by Netlogon service
- **Event 1644 NEVER fires**

Used by: ldapnomnom (TCP), cldap_ping.py (UDP), Windows DC Locator (UDP)

netlogon.log sees every username

Enable with: `nltest /dbflag:0x2080ffff`

```
# %windir%\debug\netlogon.log - EARTH-DC, MARVEL.local

# HIT - valid enabled account:
[MAILSLOT] Received ping from (null) thor on UDP LDAP
[MAILSLOT] EARTH-DC: Ping response 'Sam Logon Response Ex' thor

# MISS - account does not exist:
[MAILSLOT] Received ping from (null) magneto on UDP LDAP
[MAILSLOT] EARTH-DC: Ping response 'Sam User Unknown Ex' magneto

# DISABLED - defender sees, attacker does NOT:
[MAILSLOT] Received ping from (null) hawkeye on UDP LDAP
[MISC]      EARTH-DC: NISamVerifyUserAccountEnabled: hawkeye DISABLED
[MAILSLOT] EARTH-DC: Ping response 'Sam User Unknown Ex' hawkeye

# HONEYTOKEN - critical detection opportunity:
[MAILSLOT] Received ping from (null) svc_backup on UDP LDAP
[MAILSLOT] EARTH-DC: Ping response 'Sam Logon Response Ex' svc_backup
```

Always follow the code

THE README

"No Windows audit logs generated."

Wrong.

THE SOURCE CODE

Idapnomnom uses TCP, not UDP.

Verified.

THE EVENT LOGS

Event 5156 fires on every connection.

Confirmed.

Don't trust tool READMEs. Read the source. Verify with the logs.

The defender advantage: honeytoken gold

Disabled accounts + honeytokens

The attacker gets 'Sam User Unknown Ex' for any account that is disabled OR doesn't exist — the same response. ldapnomnom treats it as a miss. BUT netlogon.log shows **NISamVerifyUserAccountEnabled** before the response — naming the disabled account explicitly.

*Plant an ENABLED honeytoken named **svc_backup**. Any hit is the attacker.*

Caveats you need to know

- Source IP = (**null**). netlogon.log doesn't record it. Pair with Event 5156 for TCP attribution.
- Transport label says "UDP LDAP" even for TCP connections. *Format quirk — don't trust it.*
- Enable with **nltest /dbflag:0x2080ffff**. Logs to %windir%\debug\netlogon.log



// VIDEO DEMO

Get-LDAPPingDetection.ps1

Spray usernames over cLDAP — netlogon fills, 1644 stays empty.

Get-LDAPPingDetection.ps1 vs LDAP Nom Nom



VIDEO

DEMO 02 • Get-LDAPPingDetection.ps1 vs LDAP Nom Nom

Get-LDAPPingDetection.ps1 vs CLDAP



VIDEO

DEMO 03 • Get-LDAPPingDetection.ps1 vs CLDAP

CHAPTER 07

The Math Never Lies

Statistics catch tools you've never seen. Here's why it works.

Reminder: Step 0 from pre-flight

Two registry paths

```
# (1) Diagnostic logging level
HKLM\SYSTEM\CurrentControlSet\
  Services\NTDS\Diagnostics

'15 Field Engineering' = 5
# level 5 REQUIRED for 1644

# (2) Query thresholds
HKLM\SYSTEM\CurrentControlSet\
  Services\NTDS\Parameters

'Expensive Search
  Results Threshold'   = 10000
'Inefficient Search
  Results Threshold'   = 1000
'Search Time
  Threshold (msecs)'   = 30000
```

The reality

Default: **DISABLED** (level 0)

Only level 5 emits 1644 — **levels 1-4 don't**.

Default thresholds are **noisy-safe** (10k entries, 30s).
Tune lower once baseline is stable.

Event Lives in: **Directory Service log**
(not Security)

You cannot detect what you do not log. Step 0: enable 1644.

Performance impact – and how to tune it

Microsoft: 1644 is a diagnostic for expensive, inefficient, or slow searches (KB 2800945, KB 3060643).

Tune thresholds for your env

BASELINE *Safe. Quiet. Misses fast recon.*

Default (10k / 1k / 30s)

INVESTIGATE *Catches most recon tools*

Medium (500 / 100 / 1s)

HUNT *SOAPHound/BloodHound caught. CPU cost.*

Aggressive (100 / 50 / 10ms)

DEBUG *LAB ONLY. Logs everything.*

Max (1 / 1 / 1ms)

What drives the cost

- ▶ Query volume on the DC
- ▶ Lower thresholds = more events
- ▶ Event log write overhead
- ▶ Directory Service log size

No benchmark from MS. Measure in your env.

Start at INVESTIGATE. Watch DC CPU. Tighten once stable. You are the environment expert.

Four chapters of whack-a-mole

Ch 01 **OID transformations**

→ *AD rewrites it – the log never shows the string*

Ch 02 **Whitespace-agnostic regex**

→ *DC re-spaces it – wire-form rules miss*

Ch 03 **SDFlags parameter match**

→ *Change the flag – 0x4 / 0x5 / 0x7*

Ch 04 **SOAPHound (!FALSE)**

→ *Infinite nonsense attributes to choose from*

“We’ve been playing whack-a-mole with syntax for four chapters. Time for a paradigm shift.”

The paradigm shift

“Attackers need the data. Getting that data creates mathematical patterns that cannot be hidden, regardless of the tool or technique used.”

What every recon tool must do:

~5,000

Enumerate all users

~3,000

Enumerate all computers

~500

Enumerate all groups

10-30 each

Read 10-30 attrs per object

The environmental ceiling

Your AD size × your attribute density = the cost of reconnaissance.

No tool can pay that cost quietly.

Dimension 1: Volume

Example environment (illustrative):

Users: 5,234
Computers: 3,847
Groups: 892
SPNs: 127

BloodHound needs ALL of these.

Dimension 2: Density

Attrs collected per object (illustrative):

avg attrs/user: 23
avg attrs/computer: 31

Leaks as **pages/object** in 1644:

admin query: 5-20
SOAPHound: ~72 (lab capture)

Reduce volume → miss the graph. Reduce attributes → miss the attack paths. Can't dodge both.

Every 1644 tells a story: the 5 W's

Six questions turn raw counters into intent.

Question	1644 field(s)	Why it matters
WHO made the query?	Client IP, User	Attribution & targeting
WHAT are they after?	Search Filter, Returned	Attack intent
WHEN did it happen?	TimeCreated	Pattern detection
WHERE are they looking?	Base DN, Search Scope	Scope of recon
WHY does it matter?	Visited / Returned / Pages	Tool sophistication
HOW are they doing it?	Used Indexes	Query optimization

WHO + WHAT + WHERE = the target. WHY + HOW = how good they are. A whole investigation from one event.

Event 1644's secret weapon: the counters

```
# Event 1644 excerpt • SOAPHound --bhdump
# Client [::1]:59435 • User MARVEL\loki
Visited entries:
3815
Returned entries:
334
Used indexes:
DNT_index:2584:N;
Pages referenced:
24100
Pages read from disk:
0
Search time (ms):
78
```

*Every 1644 event carries performance telemetry.
Most defenders ignore it.*

Why this matters

Returned entries = what the attacker got

Visited entries = what AD had to scan ← the signal

Used indexes = DNT_index only: full table walk

High visited + no attribute index = recon. No real app queries this way.

Index reference: what 'Used Indexes' reveals

The HOW dimension, decoded — reading the 'Used Indexes' field.

Index	What it signals	Recon weight
Ancestors_index	Subtree enumeration	High
DNT_index	Full table walk, no attr index	High
idx_objectCategory	Class-based enumeration	High
pdnt_index	OU structure mapping	Medium
idx_objectClass	ADCS / class enumeration	High
idx_member	Privilege mapping	CRITICAL
idx_servicePrincipalName	Kerberoasting prep	CRITICAL
idx_sAMAccountName	Targeted account lookup	Medium

memberOf and servicePrincipalName are the loudest indexes — privilege mapping and Kerberoast prep. See them, look closer.

Your environment is your baseline

Same tool, three envs (illustrative)

SMB 200 users / 150 computers

Normal $\mu=8$ entries $\rightarrow$ SOAPHound $x=140$ ($\times 17$)

MID 5k users / 3k computers

Normal $\mu=57$ entries $\rightarrow$ SOAPHound $x=3,815$ ($\times 67$)

ENT 50k users / 40k computers

Normal $\mu=430$ entries $\rightarrow$ SOAPHound $x=41,200$ ($\times 96$)

The factor ($\times 17$, $\times 67$, $\times 96$) is what matters — not the raw count.

Your baseline depends on:

- ▶ **Users** *Base query volume scales linearly*
- ▶ **Computers** *BH computer enumeration floor*
- ▶ **Groups** *Group membership query depth*
- ▶ **SPNs** *Kerberoast target universe*
- ▶ **Admin-count=1** *Tier-0 enumeration size*
- ▶ **Cert templates** *ADCS ESC* enumeration size*
- ▶ **Legit tools** *SCCM / backup / scanner floors*

"You still have to understand your env." — me.

Building the baseline – 5 knobs

1	COLLECTION PERIOD 14 days (min 24-48 hrs)	<i>Capture daily + weekly cycles</i>
2	DATA POINT EntriesVisited per 1644	<i>Plus Returned / Pages / Indexes</i>
3	EXCLUSIONS SCCM • backup • scanners	<i>Their volume skews μ high</i>
4	SCOPE Per source IP • per user	<i>Global σ hides the signal</i>
5	WINDOW Rolling 14-day • Welford's	<i>Baseline drifts as env grows</i>

The one-liner

```
# Pull 14 days of 1644
$d = Get-WinEvent ...

# Compute baseline
$stats = $d |
    Measure-Object EntriesVisited `
        -Average -StandardDeviation

#  $\mu$ ,  $\sigma$  in hand.
```

Built-in. No ML stack required.

Per-IP μ and σ . Exclude known tools. Rolling 14-day window. Alert when $z > 3$.

The only formula you need: z-score

$$z = (x - \mu) / \sigma$$

x

WHAT JUST HAPPENED

*what just happened:
3,815 entries visited*

μ

WHAT'S NORMAL

*what's normal for
this user: 57 entries*

σ

HOW MUCH IT WOBBLES

*how much it normally
varies: ± 12 entries*

x, μ , σ are object counts. z has no unit — just 'wobbles from normal.' Alert at $z > 3$.

Worked example: counter $\rightarrow$ z-score

1

Build μ from 48 hours of normal

Normal sessions in my lab visit ~ 57 objects ($\mu = 57$).
That number varies by ± 12 entries ($\sigma = 12$).

2

Watch what just happened

This session: SOAPHound visited 3,815 objects ($x = 3,815$).

3

Compute how weird 'weird' is

$z = (3,815 - 57) / 12 = 313.2 \sigma$
99.7% of normal lives within 3σ . We are $\sim 100\times$ past that.

4

Why this can't be hidden

To stay under $z > 3$, SOAPHound has to touch < 93 objects.
That's not 'sneaky SOAPHound.' That's 'broken SOAPHound.'

Why $z > 3$? The 99.7% rule.

The empirical rule

In a normal distribution:

68% of values within $\pm 1\sigma$

95% of values within $\pm 2\sigma$

99.7% of values within $\pm 3\sigma$

Alerting at $z > 3$ means you alert on the 0.3% of behavior that is statistically abnormal.

Worked example

```
# lab 48-hour baseline:
μ = 57  objects visited
σ = 12  objects

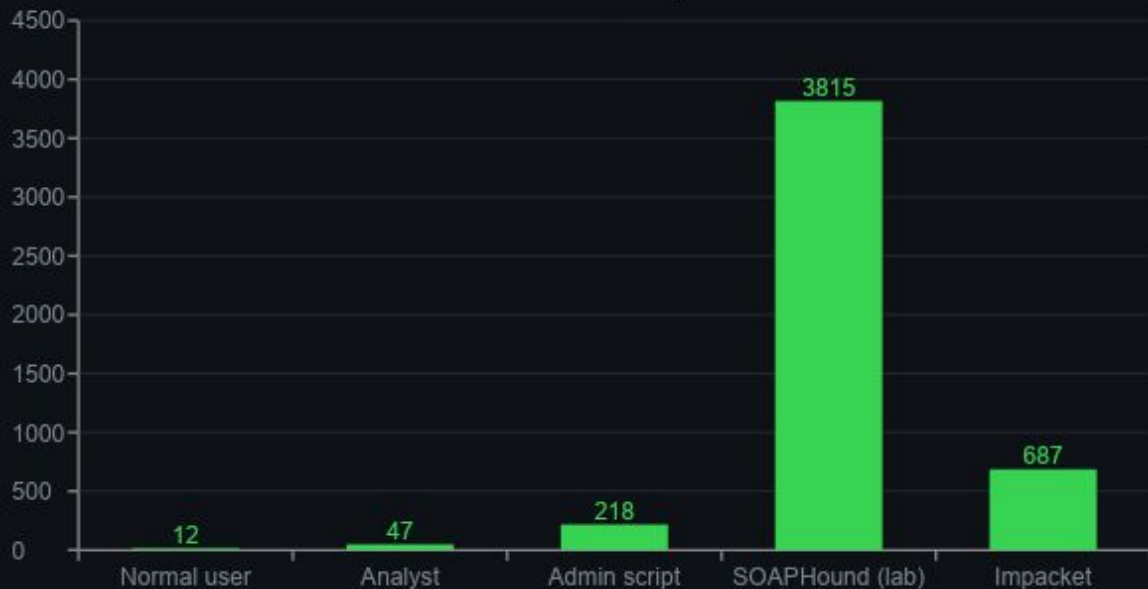
# SOAPHound session:
x = 3,815

z = (3815 - 57) / 12
z = 3758 / 12
z = 313.2 σ

# alert threshold z > 3
# we're ~100x over. ALERT.
```

The picture: baseline vs. outliers

Entries Visited per session (SOAPHound: lab · others illustrative)



THE READ

Normal user → Admin script: **one order of magnitude**

Normal user → SOAPHound: **~2.5 orders of magnitude**

*You don't need ML to see this. **You need a baseline and a calculator.***



// VIDEO DEMO

BloodHound-Detector-v2_7_5-ULTIMATE.ps1

z-score + composite probability — then the two reports.

BloodHound-Detector vs SharpHound



VIDEO

DEMO 04 · BloodHound-Detector-v2_7_5-ULTIMATE.ps1 vs SharpHound -c DCOnly

BloodHound-Detector – BloodHound-Report



VIDEO

DEMO 05 · BloodHound-Detector-v2_7_5-ULTIMATE.ps1 - BloodHound-Report

BloodHound-Detector – DFIR Report



VIDEO

DEMO 06 · BloodHound-Detector-v2_7_5-ULTIMATE.ps1 - DFIR Report

Four signals an attacker can't hide

One signal false-positives. All four at once — reconnaissance.

40% VOLUME – entries visited

How many objects AD touched per query (z-score vs baseline)

Normal: $z < 2$ (within 2σ) **Recon:** $z > 3$ (rare)
→ Attackers need the data. Tiny queries miss attack paths.

30% EFFICIENCY – visited/returned

How much AD scanned per result (visited ÷ returned)

Normal: 5–20×, indexed **Recon:** DNT-only, any ×
→ The ratio alone is ambiguous. Read it with Used Indexes.

20% INDEX – used indexes

Which AD indexes the query hit (Ancestors / SPN / objectClass)

Normal: varied indexes **Recon:** Ancestors / DNT only
→ Subtree enumeration leaves an index fingerprint AD always logs.

10% PAGES – pages referenced

How much raw data per object (proxy for attribute volume)

Normal: 5–20 pages/object **Recon:** 50+ pages/object
→ Pulling DACL + all attributes is heavy. The pages tell on you.

CHAPTER 08

Low & Slow Evasion

Patient attackers stay under every threshold. Event 1644 detects tools, not tradecraft.

Event 1644 only logs expensive queries

Three independent thresholds. Trip ANY one and it logs. Trip none, you're invisible:

Volume / Efficiency / Time — entries visited, <10% returned, or search-time ms

Defaults are huge — labs drop them to ~100 / ~50 / ~10ms to surface activity

secretsdump scopes which accounts to pull with -ldapfilter. Bulk scope is loud; targeted scope is silent.

```
# bulk DCSync scoping — caught, but ONLY by the time threshold:
secretsdump -ldapfilter (objectClass=user) → 57 ret, 46ms → LOGGED
# not expensive, not inefficient — just slow enough. one signal.

# targeted DCSync scoping — crosses ZERO of three:

secretsdump -ldapfilter (memberOf=Domain Admins) → 4 ret, 0ms → NO 1644
# 4 results, indexed, 0ms. Then it DCSyncs just those. 4662 catches that.
```

The chicken-and-egg – and the fix

The attacker's problem

To query individually you need the username list first. And to get the list, you query bulk — which trips 1644.

Unless they have the list — or skip Domain entirely →

The defender's fix

Connection bursts. ldapnomnom opens 8 parallel connections; sprayers reconnect constantly. 5156 sees every one — even without 1644.

Counts connections, not queries: one bind can carry 1,000 lookups.

```
title: LDAP / ADWS connection
name: ldap_adws_conn
logsource: {product: windows, service: security}
detection:
  sel:
    event.code: 5156
    winlog.event_data.DestPort:
      - 389 # direct LDAP (Impacket, BH.py)
      - 9389 # ADWS (Get-ADUser, SOAPHound)
condition: sel
```

```
---
title: LDAP / ADWS connection burst
correlation:
  type: event_count
  rules:
    - ldap_adws_conn
  group-by:
    - winlog.event_data.SourceAddress
  timespan: 5m
condition:
  gt: 100 # connections, not queries
```

AD is three trees. You watch one.

Almost all Event 1644 monitoring is scoped to the Domain partition. Attackers know it.

Partition	Base DN	What attackers hunt here
Domain	DC=marvel,DC=local	Users, groups, ACLs, BloodHound paths
Configuration	CN=Configuration,DC=marvel,DC=local	Cert templates → ESC1-8, trusts, sites
Schema	CN=Schema,CN=Configuration,...	Class definitions – mostly noise

The attack: base-scope walk the cert templates under Configuration, one per query, sleep between. Full ADCS recon, zero Event 1644 — and no bulk phase to catch, because that path is identical in every forest.

Three axes. Thresholds fight one.

An attacker can move along three independent axes. The literature only covers the first.

Axis	What it defeats	Cost to attacker
Volume / Time	Event 1644 thresholds	Speed – smaller, slower queries
Partition	Domain-scoped monitoring + SACLs	None – if they know where to look
Scope: base vs subtree	Anomaly detectors hunting subtree	Speed – more queries

Real low-and-slow stacks all three: base-scope reads against the Configuration partition, one object at a time, with delays between.

When recon is invisible, catch the theft

4662 = the DCSync safety net

Event 4662 fires on DS-Replication-Get-Changes-All. It catches the credential theft — not the recon.

NOT a recon detector. The enumeration itself stays silent.

Stop building single-event rules

4624 logon + 1644 targeted query + 4662 replication, same source, inside 60 seconds.

Near-impossible to fake. No single event has to cross a threshold.

```
# the sequence that is almost certainly an attack
T+0s  4624  - network logon from 10.1.1.13
T+5s  1644  - targeted query: Domain Admins membership
T+10s 4662  - DS-Replication-Get-Changes-All
T+15s 4662  - replication: krbtgt, Administrator, DA

# correlate: EventCode IN (4624, 1644, 4662)
#   by src_ip, span=60s, where event_types >= 3
# no single event is actionable. the pattern is.
```

CHAPTER 09

Obfuscation Blind Spot

Event 1644 catches the loud. LDAPMon catches the quiet. You need both.

MaLDAPtive vs Event 1644: ~70% blind

Test suite: 20 obfuscation techniques. Log source comparison:

Technique	Event 1644	LDAPMon ETW
Hex-encoded values (<code>\75ser = user</code>)	DECODED	RAW – seen
Null byte injection (<code>\00</code>)	NOT LOGGED	RAW – seen
OID attribute names (<code>1.2.840...1.4.1</code>)	NOT LOGGED	RAW – seen
Whitespace padding / newlines	NOT LOGGED	RAW – seen
Random case (<code>ObjectClass / CN</code>)	NOT LOGGED	RAW – seen
Double negation (<code>(!(x=y))</code>)	LOGGED	RAW – seen
Redundant parens (<code>((((x=y))))</code>)	NOT LOGGED	RAW – seen
Garbage filter (<code>((&(x)(invalidAttr=...)))</code>)	NOT LOGGED	RAW – seen

Double negation survives (forces a full scan). The other ~70% vanish three ways: DC normalizes · indexed ≠ expensive · optimizer drops it.



// VIDEO DEMO

Invoke-MaLDAPtiveTests.ps1

I built this expecting Event 1644 to catch most of them. Place your bet before the reveal.

MaLDAPtive vs Get-DCLDAPLogs.ps1



VIDEO

DEMO 07 · Invoke-MaLDAPtiveTests.ps1 vs Get-DCLDAPLogs.ps1

I tested all 20 – here's what 1644 missed

MaLDAPtive techniques (Bohannon & Elezaj) plus Charlie Clark's combined technique. How Event 1644 handles each:

Key	Technique	Event 1644 status
Z	Prepend zeros to numbers	Not logged
N	Names to ANR	Logged but undetectable
x	Extensible match empty rule	Not logged
D	DeMorgan transform	Not logged
A	Approximate match ~=	Logged as-is (~=)
G	Garbage filter	Not logged
s	Substring split	Survives as cn=jo*hn
W	Whitespace injection	Not logged
C	Case randomization	LDAPMon only
O	OID attribute names	LDAPMon only
X	Hex value encoding	LDAPMon only – rule exists
I	Equality to range (>=<=)	Logged as-is (>=)

Most never reach Event 1644 — and the ones that do lose their obfuscation. LDAPMon sees all 20 raw on the wire — the fix, next slide.

LDAPMon – client ETW sees the raw wire

The architecture

Provider: Microsoft-Windows-LDAP-Client

GUID: 099614a5-5dd7-4788-8bc9-e29f43db28fc

Captures: raw LDAP filter BEFORE the DC sees it. Obfuscation intact.

Tool: LDAPMon by Jonny Johnson · github.com/jonny-jhnson/LDAPMon

```
# Enable per process with registry key:
HKLM\System\CurrentControlSet\Services\ldap\Tracing\<ProcessName.exe>

# Start ETW trace:
logman start LDAPMon -p Microsoft-Windows-LDAP-Client 0x1a59afa3 0xff -o ldap.etl -ets

# Convert to events with Event ID 30 = LDAP SearchRequest
tracertp ldap.etl -o ldap.xml
```

MaLDAPtive vs LDAPMon



VIDEO

DEMO 08 · Invoke-MaLDAPtiveTests.ps1 vs LDAPMON & Get-LDAPClientLogs.ps1

Sigma rules LDAPMon can carry

Deploy via LDAPMon by Jonny Johnson • github.com/jonny-jhnson/LDAPMon

```
title: Hex-Encoded LDAP Filter
logsource:
  service: ldap-client
detection:
  sel:
    EventID: 30
    Filter|re: '\\[0-9a-fA-F]{2}'
  filter_lsass:
    ProcessName: lsass.exe
  condition: sel and not filter_lsass
```

```
title: Null Byte Injection
logsource:
  service: ldap-client
detection:
  sel:
    EventID: 30
    Filter|contains: '\\00'
  condition: sel
level: high
```

```
title: OID Attribute Notation
logsource:
  service: ldap-client
detection:
  sel:
    EventID: 30
    Filter|re: '\\(1\\.2\\.840\\.113556'
  condition: sel
```

```
title: Double Negation # ship as 2 rules
detection:
  sel_etw:
    EventID: 30
    Filter|re: '\\(\\s*!\\s*\\(\\s*!'
  sel_1644:
    event.code: 1644
    winlog.event_data.LDAPFilter|re: '\\(\\s*!\\s*\\(\\s*!'
  condition: 1 of sel_*
```

1644 cheatsheet: read every column

Field	What it reveals	Attacker footprint	Where in talk
Client IP	WHO ran the query	::1 means ADWS proxy	Ch 5A • 5B
Search Filter	WHAT they're after	(!FALSE), bitwise &, expanded DNs	Ch 01 • 04
Search Scope	Scope of search	Subtree = full enum	Ch 07
AttributeList	Fields they pulled	nTSecurityDescriptor = ACL recon	Ch 03
Controls	Server-side flags	SDFlags 0x4/0x5/0x7 OR nTSecurityDescriptor	Ch 03 • 04
Visited / Returned	How much data extracted	High visited + no attr index = recon	Ch 07

Six fields. Nine chapters. One log source. Read every column.

The forest: every blind spot, every fix

Part	Blind spot	Fix
01	OID transformations break signatures	Regex the normalized form
02	DC rewrites the filter; wire-form rules miss	Regex the logged form
03	SDFlags is invisible to string rules	Parse Controls field
04	SOAPHound !FALSE dumps without signature	Pattern-based rule
5A/B	ADWS hides source IP behind ::1	Event 5156 port join + 1138/1139 chain
06	LDAP Ping never reaches the LDAP engine	netlogon.log + 5156
07	Unknown tools evade all signatures	Statistical z-score
08	Low-and-slow stays under every 1644 threshold	Partition-aware monitoring + 4662 correlation
09	~70% of obfuscation produces no 1644	LDAPMon ETW

No single log source is complete. Layer three: Event 1644 + Event 5156 + LDAPMon ETW.

Three tools. Know when to reach for each.

SIGNATURES

fast, specific, brittle

USE WHEN

Known OIDs, tool strings, fixed patterns

AVOID WHEN

Anything rename or whitespace breaks

BEHAVIOR

patterns over strings

USE WHEN

SDFlags $\in \{0x4, 0x5, 0x7\}$, (!FALSE), nTSD
in attrs

AVOID WHEN

*When you don't understand the
protocol yet*

STATISTICS

catches the unknown

USE WHEN

EntriesVisited/Returned, volume
z-scores

AVOID WHEN

When you can't build a clean baseline

Six things to take home

- 1 Event 1644 logs the NORMALIZED filter. Rules match what AD writes, not what's sent.
- 2 Whitespace, case, hex, OID tricks vanish before 1644. Only client ETW sees them.
- 3 Server-side controls (SDFlags) and query patterns (!!FALSE)) reveal intent.
- 4 EntriesVisited / EntriesReturned is the most underused field in Windows auditing.
- 5 ADWS attribution isn't impossible. Event 5156 port-matching solves it.
- 6 Event 1644 + Event 5156 + LDAPMon ETW. That's the stack. Deploy all three.

Don't stare at one tree. Walk the forest.



Questions

(and arguments about whether regex counts as statistics)

@4ndr3w6S · huntress.com/authors/andrew-schwartz · github.com/4ndr3w6